**UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS**
Facultad de Ingeniería – Proyecto Curricular de Ingeniería de Sistemas
Grupo de Investigación Multimedia Interactiva y Animación Digital

# INFORME TÉCNICO DE AVANCE Y DESARROLLO

## HITO 4: Persistencia Mutable, Modo Portable, Patrón Plantilla y Soporte Invertido

Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox

| | | | |
|---|---|---|---|
| **Proyecto:** | AR-Sandbox: Seguimiento de Rutas | **Periodo:** | 2026-III |
| **Hito / Avance:** | Hito 4 de 7 | **Fecha de corte:** | 20 de octubre, 2026 |
| **Presentado por:** | Joshoa Alarcón Sánchez (20221020013) | **Repositorio Base:** | `YoshiK13/Magic-Sand-Expanded` |

## 1. Objetivos del Hito de Entrega

El presente documento expone las soluciones de ingeniería y la reestructuración arquitectónica correspondientes al **Hito 4**. Este ciclo se enfoca en superar las restricciones de almacenamiento impuestas por el sistema de archivos de sólo lectura de los paquetes portables *AppImage*, estandarizar la lógica de los controladores de juego mediante patrones de diseño orientados a objetos, habilitar la operación física en configuraciones invertidas (*inverted setups*) y documentar los desafíos técnicos superados durante el proceso de estabilización.

Los objetivos específicos correspondientes a este hito son:

- Diseñar e implementar una jerarquía robusta de resolución de rutas para datos mutables en entornos Linux, resolviendo la inmutabilidad de los paquetes *AppImage* mediante el almacenamiento primario en el directorio de usuario (`~/Magic-Sand/data`) y proveyendo un **Modo Portable** autónomo activable al situar una carpeta `data/` adyacente al ejecutable.
- Implementar un mecanismo de aprovisionamiento y sincronización no destructiva (*safe provisioning*) que garantice la integridad de configuraciones XML vitales (calibración homográfica, mapas hipsométricos y perfiles de visualización) sin sobrescribir las modificaciones previas del operador.
- Refactorizar la arquitectura multijuego mediante el **Patrón de Diseño Plantilla (*Template Method Pattern*)**, formalizando la clase abstracta `BaseGameController` para desacoplar el esqueleto invariable de renderizado y transformaciones espaciales respecto a la lógica específica de cada juego (`MapGameController` y `BoidGameController`).
- Integrar el soporte integral para montajes físicos invertidos (**Modo *Inverted***), permitiendo rotar el espacio de proyección 180° por software ante restricciones de montaje del proyector o del sensor Kinect, manteniendo la sincronía óptica entre la superficie de arena, la simulación de entidades y la interfaz de usuario (HUD).
- Desacoplar la orientación de la ventana de control del operador respecto a la ventana proyectada, garantizando que los avisos informativos y paneles de configuración permanezcan orientados verticalmente (*upright*) para el usuario mientras la escena sobre la arena acompaña la geometría tangible.
- Diagnosticar y mitigar los problemas críticos de implementación surgidos: sensibilidad a mayúsculas y minúsculas en rutas de Linux, colisiones lógicas por doble inversión (operación XOR errónea entre rotación de hardware y software), y contaminación de matrices en la pila gráfica de OpenGL.
- Verificar experimentalmente la arquitectura resultante mediante la ejecución y calibración del juego interactivo de boids (`BoidGameController`) sobre la estación física de arena en configuración invertida.

## 2. Arquitectura de Persistencia: Gestión de Datos Mutables y Modo Portable

### 2.1. El Dilema del Sistema de Archivos Inmutable en AppImage

El estándar *AppImage* empaqueta la aplicación dentro de una imagen de disco comprimida bajo el sistema de archivos **SquashFS**. Al momento de su ejecución, el runtime monta dinámicamente dicha imagen en una ruta temporal del anfitrión (generalmente en `/tmp/.mount_XXXXXX`) en modo de **sólo lectura** (*Read-Only*).

En la versión original heredada de `Magic-Sand`, el código asumía que el directorio `data/` se encontraba en la misma ruta física del binario compilado y que contaba con permisos irrestrictos de lectura y escritura. En consecuencia, el sistema intentaba escribir de manera recurrente:

- Matrices de calibración homográfica y límites espaciales en `data/settings/calibration.xml` y `data/settings/kinectProje`
- Mapas de elevación y parámetros de sombreado en `data/settings/sandSurfaceRendererSettings.xml`.
- Puntuaciones históricas en `data/mapGame/Scores/`.
- Volcados y mapas binarios de segmentación en carpetas de depuración (`DebugFiles/`).

Al ejecutarse como paquete portable, cualquier intento de escritura en estas rutas provocaba un error crítico del sistema de archivos (`EROFS: Read-only file system`), impidiendo salvar calibraciones y corrompiendo la experiencia del usuario.

## 2.2. Evolución y Selección del Directorio de Datos en el Entorno Linux

Para solventar esta limitación, se analizaron y desarrollaron tres fases evolutivas de almacenamiento:

### 2.2.1. Fase 1: Persistencia Oculta bajo Estándar XDG (`~/.config/Magic-Sand/data`)

Inicialmente se canalizó la redirección hacia el estándar XDG (`$XDG_CONFIG_HOME/Magic-Sand/data`). Sin embargo, esta aproximación evidenció graves desventajas operativas: en despliegues reales, docentes y operadores sin conocimientos avanzados de Linux tenían dificultades para acceder a una carpeta oculta (prefijada con un punto) con el fin de consultar imágenes de referencia, ajustar archivos XML de mapas o respaldar puntuaciones.

### 2.2.2. Fase 2: Ubicación Estándar y Visible en Home (`~/Magic-Sand/data`)

Se redefinió la ruta predeterminada en el espacio de usuario visible: `${HOME}/Magic-Sand/data/`. De este modo, el usuario puede inspeccionar, editar y respaldar directamente los datos de calibración sin requerir comandos de consola ni alterar configuraciones de visibilidad del explorador de archivos.

### 2.2.3. Fase 3: Incorporación del Modo Portable de Cero Configuración

Para exhibiciones itinerantes, laboratorios compartidos o eventos académicos donde el software se almacena y ejecuta directamente desde una memoria USB o disco de estado sólido externo, era imprescindible evitar que las calibraciones quedaran ancladas al computador anfitrión.

Se diseñó entonces el **Modo Portable**: si el script de arranque detecta la existencia de una carpeta llamada `data/` o `Magic-Sand-Data/` inmediatamente contigua al archivo ejecutable `.AppImage` (o en el directorio de trabajo activo del shell, `$OWD`), la aplicación selecciona dicha carpeta como raíz de persistencia de manera prioritaria. De esta forma, todas las calibraciones, partidas y capturas viajan de forma autocontenida en la unidad extraíble sin modificar el sistema operativo huésped.

## 2.3. Jerarquía de Resolución de Rutas en `AppRun`

La lógica de selección de la carpeta de datos mutables se formalizó en el script de entrada `AppRun` bajo la siguiente jerarquía de prioridades:

```
# Determinacion de la ruta del ejecutable en el sistema anfitrion
if [ -n "$APPIMAGE" ]; then
    APPIMAGE_DIR="$(dirname "$(readlink -f "$APPIMAGE")")"
elif [ -n "$OWD" ]; then
    APPIMAGE_DIR="$OWD"
else
    APPIMAGE_DIR="$(dirname "$(readlink -f "$0")")"
fi

# Prioridad 1: Variable de entorno manual (MAGIC_SAND_DATA u OF_DATA_PATH)
# Prioridad 2: Modo Portable (si existe 'data' o 'Magic-Sand-Data' junto al AppImage)
# Prioridad 3: Modo Estandar en Home (~/Magic-Sand/data)

if [ -n "$MAGIC_SAND_DATA" ]; then
    DATA_ROOT="$MAGIC_SAND_DATA"
elif [ -n "$OF_DATA_PATH" ]; then
    DATA_ROOT="$OF_DATA_PATH"
elif [ -d "$APPIMAGE_DIR/data" ]; then
    DATA_ROOT="$APPIMAGE_DIR/data"
elif [ -d "$APPIMAGE_DIR/Magic-Sand-Data" ]; then
    DATA_ROOT="$APPIMAGE_DIR/Magic-Sand-Data"
elif [ -n "$OWD" ] && [ -d "$OWD/data" ]; then
    DATA_ROOT="$OWD/data"
```

```
elif [ -n "$OWD" ] && [ -d "$OWD/Magic-Sand-Data" ]; then
    DATA_ROOT="$OWD/Magic-Sand-Data"
else
    DATA_ROOT="${HOME:-/home/$USER}/Magic-Sand/data"
fi
```

Listing 1: Algoritmo jerárquico de selección de datos mutables en AppRun.

### 2.4. Aprovisionamiento No Destructivo y Compatibilidad con openFrameworks

Una vez fijada la variable `DATA_ROOT`, el script ejecuta un protocolo de preparación de datos:

1. **Creación de la Jerarquía de Subdirectorios:** Se crean preventivamente los directorios requeridos por los módulos: `settings`, `colorMaps`, `mapGame/Scores`, `mapGame/ReferenceData`, `mapGame/DebugFiles`, `boidGame/DebugFiles` y `DebugFiles`.
2. **Sincronización Segura (*Safe Copy / No-Clobber*):** Los recursos maestros (plantillas iniciales, shaders y mapas por defecto) ubicados en la partición de solo lectura (`$HERE/usr/share/magic-sand/data`) se replican hacia `DATA_ROOT` usando la instrucción `cp -rn`. El modificador `-n` garantiza que jamás se sobrescriban las calibraciones o perfiles previamente afinados por el operador.
3. **Verificación Individual de Archivos Vitales:** Se inspecciona que los archivos esenciales de configuración (`calibration.xml`, `kinectProjectorSettings.xml`, `sandSurfaceRendererSettings.xml`, etc.) existan y cuenten con tamaño mayor a cero bytes; en caso de ausencia o corrupción, se reinstancia la plantilla predeterminada.
4. **Conmutación del Directorio de Trabajo (`cd`):** Dado que el núcleo de *openFrameworks* utiliza internamente rutas relativas de acceso a recursos (e.g., `ofFile("data/...")`), se ejecuta `cd "$DATA_CONTAINER"`, donde `DATA_CONTAINER` corresponde al padre de `data/`. Esto logra compatibilidad inmediata y transparente sin requerir la refactorización invasiva de cientos de llamadas internas de la librería base.

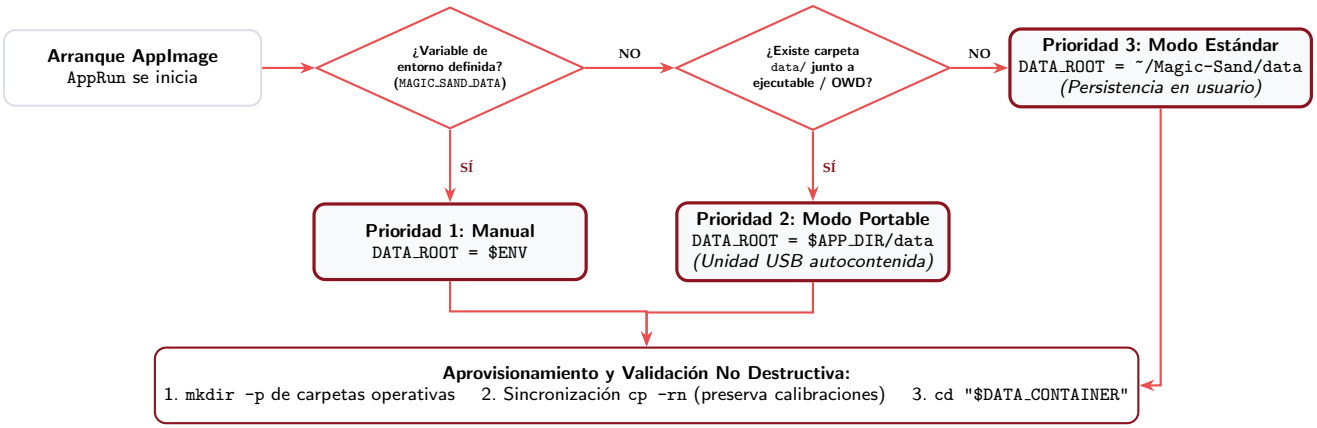



Figura 1: Diagrama de decisión y aprovisionamiento de datos mutables: jerarquía de prioridades entre variables de entorno, modo portable en memorias USB y persistencia estándar en el directorio de usuario.

## 3. Estandarización Arquitectónica: Implementación del Patrón Plantilla

### 3.1. Problema Arquitectónico Previo: Duplicación y Acoplamiento en Controladores

En las iteraciones previas del código fuente, cada minijuego (`MapGameController` y `BoidGameController`) se implementaba como una clase aislada y monolítica. Esto producía severas deficiencias de diseño:

1. **Duplicación del Algoritmo de Renderizado:** Ambas clases implementaban de forma redundante las rutinas para asociar punteros al objeto calibrador `KinectProjector`, gestionar transformaciones entre coordenadas del sensor y del proyector, e interactuar con la ventana secundaria del proyector.
2. **Inconsistencia en el Control de Estados:** El tratamiento del estado de inactividad (`isIdle()`) se manejaba con lógicas dispersas, dando lugar a renderizados residuales o parpadeos cuando un juego finalizaba y el sistema retornaba a la simulación hipsométrica estándar de la arena.

3. **Rigidez ante Transformaciones Globales:** La introducción de una inversión de coordenadas (180°) obligaba a modificar manualmente cada función de dibujo en cada controlador de manera fragmentada, elevando exponencialmente la probabilidad de regresiones de software.

### 3.2. Fundamentación Teórica del Patrón Plantilla (*Template Method Pattern*)

El patrón de diseño de comportamiento **Plantilla** (*Template Method*), formulado por el *Gang of Four (GoF)* [1], define el esqueleto invariable de un algoritmo dentro de un método en una clase base abstracta, postergando la definición de pasos específicos a las subclases derivadas. De este modo, las subclases redefinen partes puntuales del flujo de ejecución sin alterar la estructura global del algoritmo.

### 3.3. Estructura de la Clase Base Abstracta `BaseGameController`

Se formalizó la clase abstracta `BaseGameController` (`src/Games/BaseGameController.h`), la cual unifica los atributos comunes (resoluciones del proyector y Kinect, regiones de interés *ROI*, puntero al proyector, bandera de depuración) y define los dos métodos plantilla centrales:

#### 3.3.1. 1. Método Plantilla para Renderizado en el Proyector (`drawProjectorWindow`)

Define el algoritmo invariable de dibujo proyectado sobre la arena física:

```
virtual void drawProjectorWindow() {
    drawScene();
    if (!isIdle()) {
        if (isInverted()) {
            ofPushMatrix();
            ofTranslate(projRes.x, projRes.y);
            ofRotate(180.0f);
            drawHUD();
            ofPopMatrix();
        } else {
            drawHUD();
        }
    }
    ofSetColor(255, 255, 255);
}
```

Listing 2: Método plantilla para el renderizado sobre el videoproyector (BaseGameController.h).

El algoritmo garantiza invariablemente que:

- La escena base (`drawScene()`) siempre se renderiza primero en las coordenadas espaciales del mundo de arena.
- Si el juego no está inactivo, el HUD perimetral (`drawHUD()`) se dibuja aplicando la rotación de 180° de forma centralizada si el modo invertido está activo.
- El estado cromático de OpenGL se restaura obligatoriamente a blanco puro (`ofSetColor(255, 255, 255)`), previniendo contaminación visual en passes subsecuentes.

#### 3.3.2. 2. Método Plantilla para la Previsualización del Operador (`drawMainWindow`)

Resuelve la disparidad de orientación entre la mesa tangible y el operador humano:

```
virtual void drawMainWindow(float x, float y, float w, float h) {
    // 1. Elementos de escena rotan solidariamente con la superficie de arena
    ofPushMatrix();
    ofTranslate(x + w / 2.0f, y + h / 2.0f);
    if (isInverted()) {
        ofRotate(180.0f);
    }
    drawPreviewScene(-w / 2.0f, -h / 2.0f, w, h);
    ofPopMatrix();

    // 2. Capa de HUD permanece SIEMPRE vertical y legible para el operador
    drawPreviewHUD(x, y, w, h);

    ofSetColor(255, 255, 255);
}
```

Listing 3: Método plantilla para la pantalla del operador (BaseGameController.h).

#### 3.3.3. Operaciones Primitivas y Métodos Gancho (*Hooks*)

La clase define métodos virtuales protegidos que actúan como ganchos implementados por las subclases:

- `virtual void drawScene() = 0 / {}`: Renderizado de entidades del juego en la arena (e.g., bancos de peces, conejos, mapas topográficos).
- `virtual void drawHUD() = 0 / {}`: Renderizado de textos, temporizadores y puntajes en el proyector.
- `virtual void drawPreviewScene(float x, float y, float w, float h)`: Adaptación de la escena física a la ventana del operador.
- `virtual void drawPreviewHUD(float x, float y, float w, float h)`: Elementos informativos mostrados al operador en orientación vertical normal.
- **Métodos Utilitarios Auxiliares:** `isInverted()`, `kinectToProj()`, `getElevation()`, `getOrientedAngle()` y `getOrientedX()`.

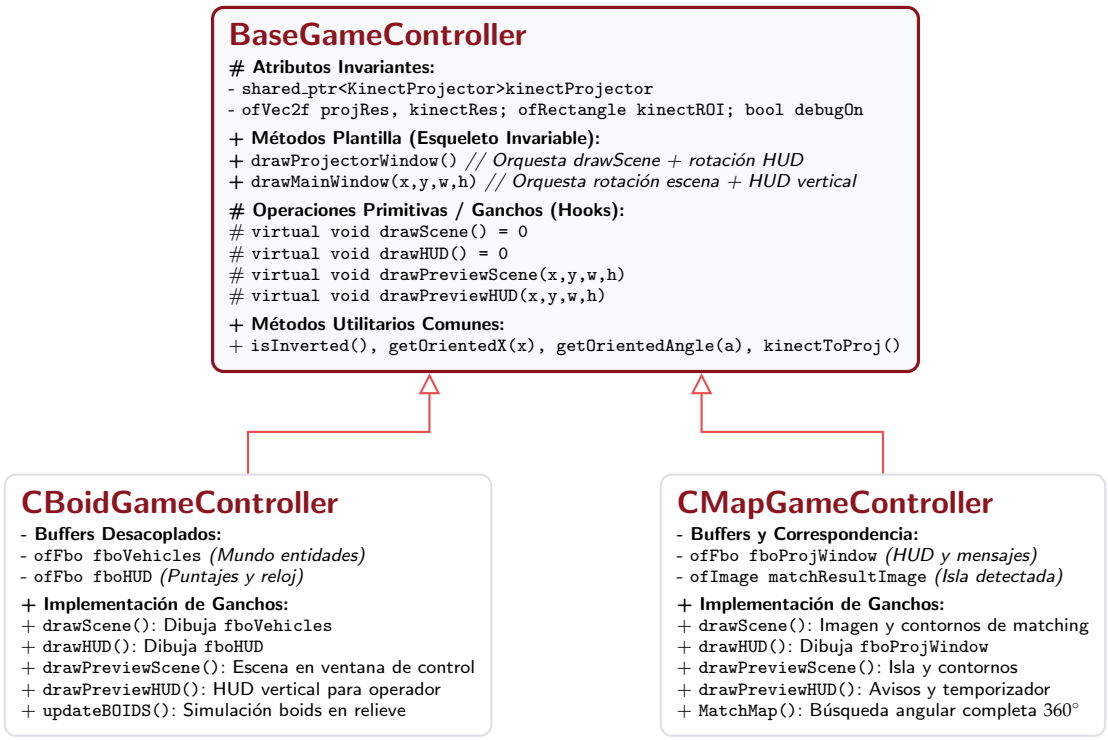



Figura 2: Diagrama de clases del Patrón Plantilla aplicado a la plataforma multijuego: la clase abstracta `BaseGameController` encapsula el flujo de renderizado y las transformaciones espaciales, mientras las subclases concretas implementan la lógica particular de simulación.

## 4. Diseño e Integración del Modo Ïnverted"(Setup Físico Invertido)

### 4.1. Motivación Física y Ergonómica en el Montaje

En instalaciones físicas de realidad aumentada sobre cajas de arena, las restricciones espaciales del recinto, el tipo de soporte estructural (trípodes de carga, trusses o marcos de perfilería de aluminio) y la longitud del tiro focal del proyector imponen frecuentemente limitaciones mecánicas severas.

En múltiples escenarios experimentales, el videoproyector debe montarse en sentido opuesto al sensor Kinect para evitar oclusiones de sombra o acomodar la salida de cables hacia la estación de cómputo. Asimismo, la ubicación del público respecto a la consola del operador suele quedar invertida 180°. Resolver esto por vía mecánica exige desmontar estructuras pesadas o rotar cajas que contienen entre 50 y 100 kg de arena cinética.

Para otorgar versatilidad absoluta de instalación, se diseñó e implementó la función **Inverted** mediante software, permitiendo invertir completamente la orientación visual de la proyección y los juegos mediante un simple interruptor en la interfaz gráfica.

### 4.2. Integración en la Cadena de Calibración y Persistencia

1. **Control Gráfico en GUI:** Se añadió un interruptor (*Toggle*) titulado "Inverted" en el panel principal `ofxDatGui` dentro de `KinectProjector`.
2. **Persistencia en XML:** El estado de la bandera se almacena en `settings/kinectProjectorSettings.xml` mediante el nodo `<INVERTED>0/1</INVERTED>`, preservando la configuración elegida entre reinicios de la aplicación.
3. **Propagación Jerárquica:** El método de acceso `kinectProjector->getIsInverted()` suministra el estado booleano a todos los módulos: `SandSurfaceRenderer`, `BaseGameController` y sus subclases.

### 4.3. Transformaciones Matemáticas en el Pipeline Gráfico

Bajo un montaje convencional, un punto tridimensional en el espacio de la arena $\mathbf{P}_{\text{arena}} = (X, Y, Z)^T$ se mapea al espacio 2D de coordenadas de pantalla del proyector $\mathbf{p}_{\text{proj}} = (u, v)^T$ mediante la matriz de homografía estimada durante la calibración:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \sim \mathbf{H}_{3\times 3} \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix} \tag{1}$$

Cuando el modo invertido se encuentra activo, la escena completa debe someterse a una rotación rígida bidimensional de 180° ($\pi$ rad) alrededor del centro del proyector, o equivalentemente, una reflexión central respecto a las dimensiones de proyección $(W_{\text{proj}}, H_{\text{proj}})$:

$$\begin{pmatrix} u_{\text{inv}} \\ v_{\text{inv}} \\ 1 \end{pmatrix} = \begin{pmatrix} \cos(\pi) & -\sin(\pi) & W_{\text{proj}} \\ \sin(\pi) & \cos(\pi) & H_{\text{proj}} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} -1 & 0 & W_{\text{proj}} \\ 0 & -1 & H_{\text{proj}} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \tag{2}$$

lo que produce analíticamente:

$$u_{\text{inv}} = W_{\text{proj}} - u, \qquad v_{\text{inv}} = H_{\text{proj}} - v \tag{3}$$

En la práctica sobre OpenGL, esta transformación se inyecta en la pila de matrices del renderizador:

```
ofTranslate(projRes.x, projRes.y);  ofRotate(180.0f);
```

Adicionalmente, se programaron funciones de mapeo analítico para elementos vectoriales en `BaseGameController`:

- **Alineación Horizontal de HUD (`getOrientedX`):**

$$x_{\text{orient}} = \begin{cases} W_{\text{proj}} - x, & \text{si } \texttt{isInverted()} \\ x, & \text{en caso contrario} \end{cases} \tag{4}$$

- **Orientación Angular de Agentes y Boids (`getOrientedAngle`):**

$$\theta_{\text{orient}} = \begin{cases} \theta + 180°, & \text{si } \texttt{isInverted()} \\ \theta, & \text{en caso contrario} \end{cases} \tag{5}$$

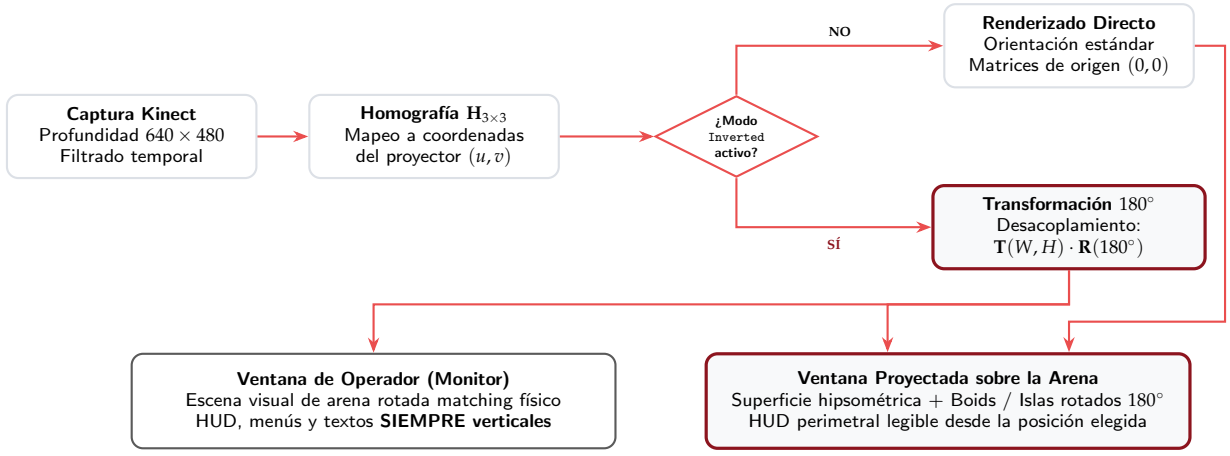



Figura 3: Pipeline de transformación geométrica y desacoplamiento espacial: la escena proyectada y la arena rotan solidariamente 180° en modo invertido, mientras que la interfaz del operador preserva la orientación natural vertical.

### 4.4. Desacoplamiento de FBOs: Escena frente a HUD en `BoidGameController`

Inicialmente, `BoidGameController` volcaba la simulación de boids y los textos de conteo y puntajes sobre un único *Framebuffer Object* (`fboVehicles`). Al aplicar la transformación invertida sobre dicho buffer, los caracteres tipográficos del HUD resultaban boca abajo e invertidos horizontalmente, volviéndose completamente ilegibles.

La solución de ingeniería consistió en desacoplar el renderizado en dos FBOs independientes:

1. `fboVehicles`: Almacena exclusivamente los agentes móviles virtuales (peces y conejos) y elementos del mundo, renderizados en sincronía con el relieve de la arena.
2. `fboHUD`: Almacena los textos de conteo regresivo, etiquetas de jugadores (*Player 1 / Player 2*), línea divisoria de la cancha y puntajes numéricos.

Con esta separación, las posiciones de los textos se calculan utilizando `getOrientedX()` para alternar simétricamente los lados de la cancha entre jugadores, y la rotación de 180° se aplica al conjunto mediante la plantilla de la clase base sin corromper la legibilidad.

## 5. Desafíos Críticos y Problemas Encontrados en el Camino

Durante el ciclo de desarrollo e integración del Hito 4 se enfrentaron múltiples anomalías técnicas de alta complejidad que demandaron diagnósticos minuciosos a bajo nivel:

### 5.1. 1. Sensibilidad a Mayúsculas en Sistemas de Archivos Linux (Bug `BoidGame`)

- **Síntoma:** Al habilitar la opción de depuración (`DumpDebugFiles`) en `BoidGameController`, la aplicación fallaba silenciosamente o crasheaba al intentar salvar mapas binarios.
- **Causa Raíz:** El código heredado había sido concebido originalmente bajo macOS y Windows, donde los sistemas de archivos (APFS insensible a mayúsculas y NTFS) tratan `BoidGame` y `boidGame` como el mismo directorio. En Linux (sistema ext4), el sistema de archivos es estrictamente sensible a mayúsculas (*case-sensitive*). El código creaba `data/boidGame/DebugFiles/`, pero intentaba escribir en `data/BoidGame/DebugFiles/`.
- **Solución:** Se refactorizó `BoidGameController.cpp` para normalizar las rutas en minúsculas unificadas y se actualizó el script de construcción de Linux para crear deterministamente la carpeta correspondiente.

### 5.2. 2. El Error de Doble Inversión: Operación Lógica XOR entre Hardware y Software

- **Síntoma:** Al activar el modo invertido en ciertos montajes, los boids comenzaban a dibujarse con sus sprites reflejados sobre el eje vertical, nadando de espaldas o mostrando su geometría invertida respecto a la trayectoria real de movimiento.
- **Causa Raíz:** En el código temprano se implementó la bandera de renderizado de vehículos mediante una compuerta XOR:

```
doFlippedDrawing = kinectProjector->getProjectionFlipped() ^ isInverted();
```

  La función `getProjectionFlipped()` detecta si la matriz de homografía $\mathbf{H}$ tiene signo negativo en su escala horizontal ($\mathbf{H}_{0,0} < 0$), lo cual es una propiedad geométrica fija del acople óptico de los lentes. Al operar con XOR junto a `isInverted()`, cuando ambas condiciones eran verdaderas se producía $1 \oplus 1 = 0$, desactivando la corrección óptica intrínseca y generando un efecto de doble inversión espacial.
- **Solución:** Se eliminó la dependencia de `isInverted()` sobre `doFlippedDrawing`, restableciendo:

```
doFlippedDrawing = kinectProjector->getProjectionFlipped();
```

  La inversión por software fue delegada estrictamente a la matriz global del método plantilla en la clase base, manteniendo la integridad morfológica de los sprites de los vehículos.

### 5.3. 3. Contaminación de Matrices OpenGL y Desfase en Ventana de Operador

- **Síntoma:** Al activar el modo invertido y alternar entre minijuegos, la previsualización del operador se desplazaba fuera del cuadrante visible o los paneles de la interfaz gráfica GUI aparecían distorsionados con coloraciones rojas no deseadas.
- **Causa Raíz:** Existía una asimetría entre las invocaciones a `ofPushMatrix()` y `ofPopMatrix()` al conmutar estados del juego, sumado al hecho de que llamadas internas a funciones de OpenCV modificaban el color activo de OpenGL sin restaurarlo antes de retornar el control.

- **Solución:** Se encapsuló la manipulación de matrices dentro del patrón plantilla en `BaseGameController::drawMainWindow()` y se introdujo la llamada explícita `ofSetColor(255, 255, 255)` al culminar cada pase de renderizado en `ofApp.cpp`, `SandSurfaceRenderer.cpp` y en los controladores de juego.

### 5.4. 4. Búsqueda Angular Restringida en Reconocimiento de Islas (`MatchMap`)

- **Síntoma:** En `MapGameController`, cuando el usuario esculpía una isla sobre la arena en una orientación rotada 180° bajo el modo invertido, el algoritmo de correspondencia (*matching*) no reconocía la forma geográfica y asignaba puntuaciones nulas.
- **Causa Raíz:** La rutina de optimización en `MatchMap()` limitaba el ángulo de búsqueda a una ventana angular estrecha de $\pm 45°$ alrededor de un centro estático, asumiendo que el usuario siempre esculpiría la isla en una sola dirección.
- **Solución:** Se expandió el rango angular de búsqueda de la correlación a los 360° completos ($\theta \in [-180°, +180°]$) y se agregó limpieza preventiva de vectores de contorno (`MatchResultContours.clear()`), logrando una detección morfológica invariante a la rotación de la arena física.

## 6. Verificación Experimental: Juego de Boids en Setup Invertido

Con el fin de comprobar empíricamente la estabilidad de la arquitectura implementada, se efectuaron pruebas operativas en la estación física AR-Sandbox de la Universidad Distrital con el modo `Inverted` habilitado.

Se desplegó el minijuego de boids (`CBoidGameController`), evaluando los siguientes parámetros de calidad:

- **Fidelidad de Navegación Tangible:** Los agentes virtuales (peces que navegan en las zonas bajas de agua y conejos que habitan en las cotas intermedias de vegetación) respondieron instantáneamente a las modificaciones manuales de la arena. A pesar de la rotación de 180° del espacio de proyección, la detección de pendientes ($\nabla z$) y colisiones físicas operó con total correspondencia métrica.
- **Ergonomía de Interfaz y Marcadores:** La línea central divisoria se proyectó perpendicularmente en el centro exacto de la caja física, y los marcadores de puntuación para el Jugador 1 y Jugador 2 se intercambiaron simétricamente, facilitando la competencia entre dos usuarios situados a lados opuestos de la mesa.
- **Monitoreo Simultáneo del Operador:** Mientras la proyección sobre la arena se ejecutaba invertida, la consola del operador en el computador portátil mantuvo su interfaz de texto, deslizadores de parámetros y botones en orientación normal perfectamente legible.

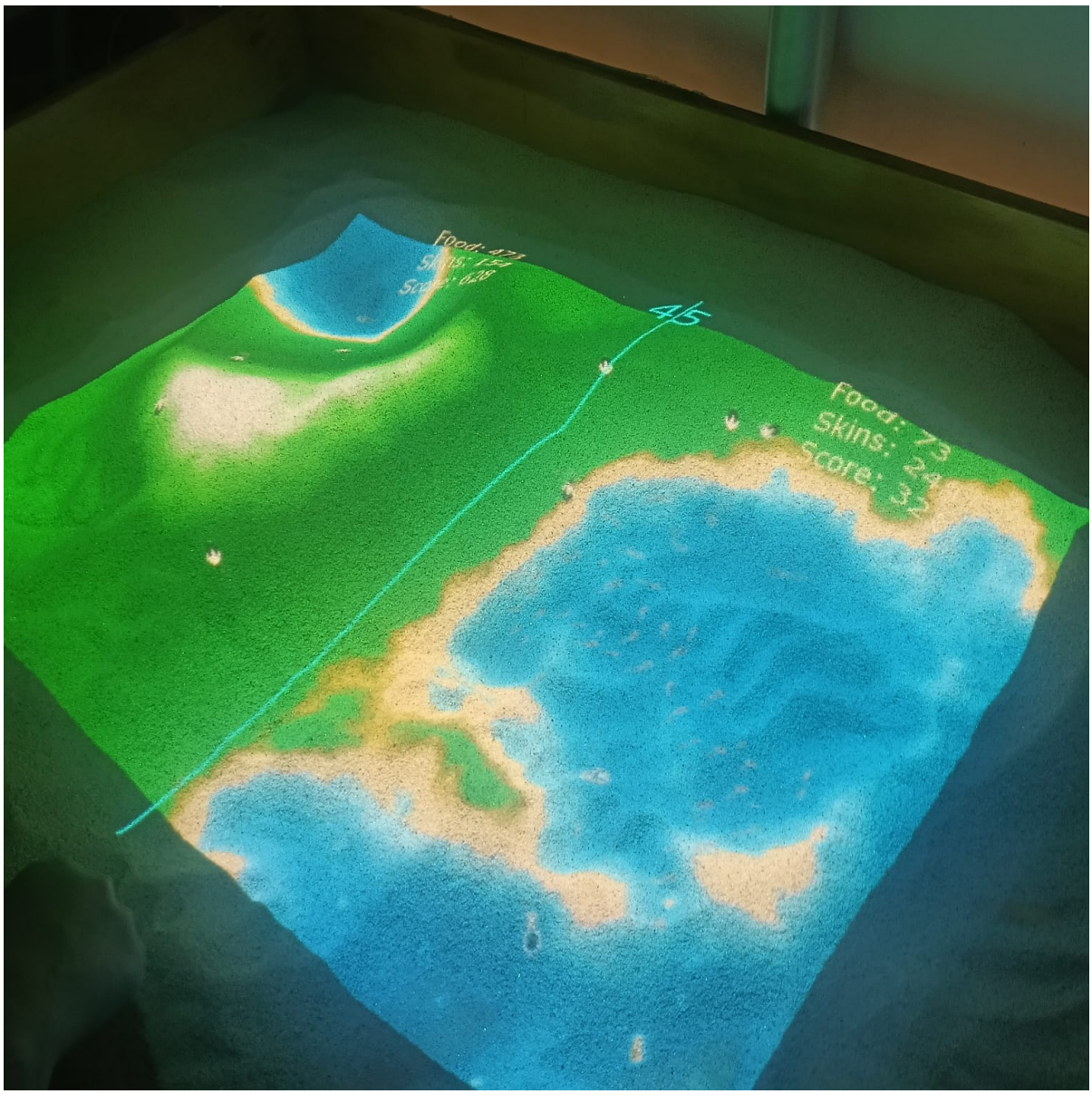



Figura 4: Verificación experimental en la plataforma física: despliegue del juego de boids bajo configuración invertida de hardware, demostrando correspondencia topográfica y desacoplamiento ergonómico del HUD.

## 7. Criterios de Aceptación y Estado del Hito 4

### 7.1. Matriz de Verificación del Hito 4

| Componente / Requisito | Criterio de Aceptación | Meta | Estado |
|---|---|---|---|
| Persistencia de Datos Mutables | Almacenamiento en `~/Magic-Sand/data` con sincronización segura | Cero pérdidas | **Conforme** |
| Soporte de Modo Portable | Activación autónoma al detectar `data/` contiguo en USB / OWD | Plug-and-Play | **Conforme** |
| Patrón de Diseño Plantilla | Clase abstracta `BaseGameController` con métodos esqueleto invariantes | 100 % desacoplado | **Conforme** |
| Modo Invertido de Hardware | Rotación de 180° por software con persistencia en XML de configuración | Sincronía 360° | **Conforme** |
| Desacoplamiento de HUD | Textos y paneles de operador verticales mientras la arena rota físicamente | Ergonomía total | **Conforme** |
| Mitigación de Bugs Críticos | Corrección de *case-sensitivity*, error XOR y estabilidad de matrices | Cero regresiones | **Conforme** |
| Desempeño en Ejecución | Tasa de refresco estable en proyección y simulación simultánea | $\geq$ 60 fps | **Conforme (60 fps)** |

### 7.2. Compromisos para el Siguiente Hito Quincenal (Hito 5)

Conforme a la planificación del proyecto de grado, las actividades prioritarias para el Hito 5 abarcan:

1. **Compromiso 1:** Culminar la implementación y refinamiento de la mecánica central del juego de laberintos y seguimiento de rutas (*TerraMaze AR*), aprovechando la clase `BaseGameController`.
2. **Compromiso 2:** Integrar el evaluador de similitud geométrica por coeficiente Dice (*DSC*) para comparar en tiempo real el canal de arena esculpido por el participante frente a la ruta óptima proyectada.
3. **Compromiso 3:** Conducir las primeras pruebas de usuario formalizadas con estudiantes para evaluar la usabilidad, tiempos de aprendizaje y retroalimentación tangible en la plataforma física.

## 8. Recursos Computacionales y Reproducibilidad

- **Entorno de Construcción y Ejecución:** Distribución Linux x86_64 (kernel 6.x+, compatible con Wayland y X11), *Docker Engine 24.x+*, paquete autocontenido `Magic-Sand-x86_64.AppImage`.
- **Comando para Ejecución Estándar:** `./Magic-Sand-x86_64.AppImage` (utiliza automáticamente `~/Magic-Sand/data`).
- **Comando para Ejecución en Modo Portable:** `mkdir -p data && ./Magic-Sand-x86_64.AppImage` (restringe toda la persistencia a la carpeta local adyacente en memorias USB).
- **Hardware de Verificación:** Portátil Intel Core i7, GPU Intel Iris / NVIDIA GTX, sensor Microsoft Kinect v1 (Modelo 1414 / K4W 1473 con alimentación externa de 12 V DC), videoproyector de tiro corto montado en estructura cenital invertida.

## 9. Referencias

## Referencias

[1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[2] Peter, S. (2018). *AppImage: Linux apps that run anywhere*. Especificación y herramientas de despliegue: https://appimage.org/.

[3] Reynolds, C. W. (1987). *Flocks, herds and schools: A distributed behavioral model*. ACM SIGGRAPH Computer Graphics, 21(4), 25-34.

[4] Wolf, T. (2016). *Magic-Sand: Real-Time Topographic Projection Mapping on Kinetic Sand*. Repositorio de código abierto en GitHub: https://github.com/thomwolf/magic-sand.

[5] Reed, K. E., et al. (2014). *Shaping the future of augmented reality in science education: The Augmented Reality Sandbox*. UC Davis W.M. Keck Center for Active Visualization in the Earth Sciences (KeckCAVES).

[6] Kogan, G. (2014). *ofxKinectProjectorToolkit: Calibration library for projector-camera systems with Kinect*. GitHub: https://github.com/genekogan/ofxKinectProjectorToolkit.

[7] OpenKinect Community (2016). *libfreenect: Open source drivers for the Microsoft Kinect*. https://openkinect.org/.

[8] openFrameworks Community (2016). *openFrameworks C++ Creative Coding Framework Documentation (v0.9.3)*. Disponible en línea: https://openframeworks.cc/.

[9] Grupo de Investigación Multimedia Interactiva y Animación Digital (2026). *Plan de Trabajo y Cronograma: Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox*. Universidad Distrital Francisco José de Caldas.